Quickstarts

Interact with contracts

Copy page

Execute contract functions on Arc Testnet to mint tokens, transfer assets, and perform contract operations.

This tutorial guides you through interacting with smart contracts deployed on Arc Testnet. You'll learn how to execute contract functions like minting tokens, transferring assets, and performing contract-specific operations for ERC-20, ERC-721, ERC-1155, and Airdrop contracts.

Prerequisites

Complete the **Deploy contracts** tutorial first. You'll need a deployed contract.

Step 1. Update your project

In this step, you update the project you created in the Deploy contracts tutorial with the additional environment variable and npm scripts needed for contract interactions.

1.1. Set environment variables

Add this new variable to your existing `.env` file (from the Deploy contracts tutorial):

.env

```
RECIPIENT_WALLET_ADDRESS=YOUR_RECIPIENT_ADDRESS
```

- `RECIPIENT_WALLET_ADDRESS` is the wallet address that receives transferred tokens during the interaction examples.

Your `.env` file should already have `CIRCLE_API_KEY`, `CIRCLE_ENTITY_SECRET`, `WALLET_ID`, `WALLET_ADDRESS`, and `CONTRACT_ADDRESS` from the Deploy contracts tutorial. You're only adding 1 new variable here.

The npm run commands in this tutorial load variables from `.env` using Node.js native env-file support.

Prefer editing `.env` files in your IDE or editor so credentials are not leaked to your shell history.

1.2. Add npm scripts

Add run scripts for contract interactions to your `package.json`:

```
npm pkg set scripts.interact-erc20="tsx --env-file=.env interact-erc20.ts"
npm pkg set scripts.interact-erc721="tsx --env-file=.env interact-erc721.ts"
npm pkg set scripts.interact-erc1155="tsx --env-file=.env interact-erc1155.ts"
npm pkg set scripts.interact-airdrop="tsx --env-file=.env interact-airdrop.ts"
```

Step 2. Interact with contracts

Select the contract type you want to interact with from the tabs below.

ERC-20 ERC-721 ERC-1155 Airdrop

Interact with ERC-20 contracts

ERC-20 tokens support standard fungible token operations. You'll learn to mint new tokens and transfer them between addresses.

Mint tokens

Use the `mintTo` function to mint tokens. The wallet must have `MINTER_ROLE`.

Node.js Python cURL

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/developer
                                                          ›
const circleDeveloperSdk = initiateDeveloperControlledWalletsClient({
  apiKey: process.env.CIRCLE_API_KEY,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET,
});

const mintResponse =
  await circleDeveloperSdk.createContractExecutionTransaction({
    walletId: process.env.WALLET_ID,
    abiFunctionSignature: "mintTo(address,uint256)",
    abiParameters: [
      process.env.WALLET_ADDRESS,
      "1000000000000000000", // 1 token with 18 decimals
    ],
    contractAddress: process.env.CONTRACT_ADDRESS,
    fee: {
      type: "level",
      config: {
        feeLevel: "MEDIUM",
      },
    },
  });

console.log(JSON.stringify(mintResponse.data, null, 2));
```

Response:

```
{
  "id": "601a0815-f749-41d8-b193-22cadd2a8977",
  "state": "INITIATED"
}
```

Token decimals: ERC-20 tokens typically use 18 decimals. To mint 1 token, use `1000000000000000000` (1 × 10^18).

Transfer tokens

Use the `transfer` function to send tokens to another address.

Node.js Python cURL

```
const transferResponse =
  await circleDeveloperSdk.createContractExecutionTransaction({
    walletId: process.env.WALLET_ID,
    abiFunctionSignature: "transfer(address,uint256)",
    abiParameters: [
      process.env.RECIPIENT_WALLET_ADDRESS,
      "1000000000000000000", // 1 token with 18 decimals
    ],
    contractAddress: process.env.CONTRACT_ADDRESS,
    fee: {
      type: "level",
      config: {
        feeLevel: "MEDIUM",
      },
    },
  });

console.log(JSON.stringify(transferResponse.data, null, 2));
```

Response:

```
{
  "id": "601a0815-f749-41d8-b193-22cadd2a8977",
  "state": "INITIATED"
}
```

Full ERC-20 interaction script

Here's the full script combining mint and transfer operations:

interact-erc20.ts interact_erc20.py

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/developer
                                                        ›
const circleDeveloperSdk = initiateDeveloperControlledWalletsClient({
  apiKey: process.env.CIRCLE_API_KEY,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET,
});

async function main() {
  // Mint tokens
  const mintResponse =
    await circleDeveloperSdk.createContractExecutionTransaction({
      walletId: process.env.WALLET_ID,
      abiFunctionSignature: "mintTo(address,uint256)",
      abiParameters: [
        process.env.WALLET_ADDRESS,
        "1000000000000000000", // 1 token with 18 decimals
      ],
      contractAddress: process.env.CONTRACT_ADDRESS,
      fee: {
        type: "level",
        config: {
          feeLevel: "MEDIUM",
        },
      },
    });

  console.log(JSON.stringify(mintResponse.data, null, 2));

  // Transfer tokens
  const transferResponse =
    await circleDeveloperSdk.createContractExecutionTransaction({
      walletId: process.env.WALLET_ID,
      abiFunctionSignature: "transfer(address,uint256)",
      abiParameters: [
        process.env.RECIPIENT_WALLET_ADDRESS,
        "1000000000000000000", // 1 token with 18 decimals
      ],
      contractAddress: process.env.CONTRACT_ADDRESS,
      fee: {
        type: "level",
        config: {
```

```
            feeLevel: "MEDIUM",
          },
        },
      });

  console.log(JSON.stringify(transferResponse.data, null, 2));
}

main();
```

Run the script:

Node.js Python

```
npm run interact-erc20
```

Summary

After completing this tutorial, you've learned how to:

- Execute contract functions using the Circle SDKs
- Mint and transfer tokens for your deployed contracts
- Perform contract-specific operations based on token type

Was this page helpful? Yes No

Access USDC crosschain Monitor contract events

Network status
Blog
Community
Arc homepage
Terms of service
Privacy policy
Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.